SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.
Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.
Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

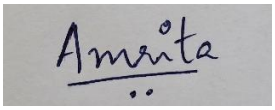
Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I AMRITA ANAIKUAMR - 192471035 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

ANSWER:

1. Servlet Architecture

A servlet-based web application follows a **client-server architecture**:

Flow:

Student → Web Browser → Web Server/Servlet Container → Servlet → Database → Response

Main components:

1. Client: Student enters details through an HTML form.
2. Web Server: Receives the HTTP request.
3. Servlet Container: Manages servlet execution and lifecycle.
4. Servlet: Processes GET/POST requests and performs business logic.
5. Database: Stores student registration information.
6. Response: Servlet sends HTML/text response back to the browser.

2. HTML Form

```
<!DOCTYPE html>
<html>
<head>
   <title>Student Registration</title>
</head>
<body>
<h2>Student Registration</h2>
<form action="StudentServlet" method="post">
   Name:
   <input type="text" name="name"><br><br>
   Email:
```

```
    <input type="email" name="email"><br><br>
    Course:
    <input type="text" name="course"><br><br>
    <input type="submit" value="Register">
</form>
</body>
</html>
```

3. Servlet Using GET and POST

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class StudentServlet extends HttpServlet {
    public void init() throws ServletException {
        System.out.println("Servlet Initialized");
    }
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
                         throws ServletException, IOException {
        response.setContentType("text/html");
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");
        PrintWriter out = response.getWriter();
        out.println("<h2>Student Details - GET</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Email: " + email + "<br>");
        out.println("Course: " + course);
    }
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
                          throws ServletException, IOException {
        response.setContentType("text/html");
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");
        PrintWriter out = response.getWriter();
```

```
        out.println("<h2>Registration Successful</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Email: " + email + "<br>");
        out.println("Course: " + course);
    }
    public void destroy() {
        System.out.println("Servlet Destroyed");
    }
}
```

If your college uses older Java EE/Tomcat versions, replace jakarta.servlet.* with javax.servlet.*.

4. Servlet Life Cycle

The servlet has three main lifecycle methods:

1. init()

- Called only once when the servlet is created.
- Used for initialization tasks such as loading configuration or establishing resources.

2. service()

- Called whenever a request is received.
- Determines whether the request is GET, POST, etc., and invokes doGet() or doPost().

3. destroy()

- Called once before the servlet is removed from memory.
- Used to release resources.

Lifecycle:

Servlet Loading → init() → service() → doGet()/doPost() → destroy()

5. GET vs POST

GET	**POST**
Data is sent in URL	Data is sent in request body
Data is visible in URL	Data is not normally displayed in URL
Suitable for retrieving data	Suitable for submitting data
Limited URL length	Can send larger amounts of data
Not suitable for passwords/sensitive information	More appropriate for sensitive form submission

Question 2:

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.
Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

ANSWER:

1. Session Management

HTTP is a stateless protocol, meaning the server does not automatically remember a user between different requests.

For example:

Login → Home Page → Profile → Shopping Cart

The application needs session tracking to identify that all these requests belong to the same user.

Servlets provide HttpSession for server-side session management and Cookies for storing small pieces of information on the client.

Session Tracking Flow

User Login

↓

Servlet verifies username/password

↓

HttpSession is created

↓

Session ID is generated

↓

Session ID is stored in a cookie (usually JSESSIONID)

↓

Browser sends session ID with subsequent requests

↓

Server identifies the user's session

↓

User remains logged in

2. Login Servlet Using HttpSession

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class LoginServlet extends HttpServlet {
   protected void doPost(HttpServletRequest request,
                         HttpServletResponse response)
                         throws ServletException, IOException {
      String username = request.getParameter("username");
      String password = request.getParameter("password");
      if (username.equals("admin") && password.equals("1234")) {
         HttpSession session = request.getSession();
         session.setAttribute("username", username);
         response.sendRedirect("home");
      } else {
         response.getWriter().println("Invalid Login");
      }
   }
}
```

3. Home Servlet

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class HomeServlet extends HttpServlet {
   protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
                        throws ServletException, IOException {
      HttpSession session = request.getSession(false);
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();
      if (session != null &&
         session.getAttribute("username") != null) {
         String username =
            (String) session.getAttribute("username");
         out.println("<h2>Welcome " + username + "</h2>");
         out.println("<p>You are logged in.</p>");
```

```
        } else {
            out.println("<h2>Please Login</h2>");
        }
    }
}
```

4. Logout

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
                         throws ServletException, IOException {
        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate();
        }
        response.getWriter().println("Logged out successfully");
    }
}
```

5. Cookies

A cookie is a small piece of data stored in the user's browser.

Creating a Cookie

```
Cookie cookie = new Cookie("username", "admin");
cookie.setMaxAge(3600);
response.addCookie(cookie);
```

Reading a Cookie

```
Cookie[] cookies = request.getCookies();
if (cookies != null) {
    for (Cookie c : cookies) {
        if (c.getName().equals("username")) {
            System.out.println(c.getValue());
        }
    }}
```

6. HttpSession vs Cookies

HttpSession	**Cookies**
Data is mainly stored on server	Data is stored on client/browser
More suitable for session information	Suitable for small preferences/identifiers
Session ID is usually sent through a cookie	Actual cookie data is sent to server
Can store objects	Stores string values
Server controls session data	User can delete/block cookies
Better for authentication state	Useful for remembering preferences

HttpSession is generally preferable for maintaining authenticated user sessions because important session information can remain on the server. Cookies are commonly used to carry the session identifier (JSESSIONID) between the browser and server. For secure applications, cookies should be configured with appropriate security attributes such as HttpOnly and Secure, and the application should use HTTPS.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application. Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

ANSWER:

JDBC Architecture

Java Application

↓

JDBC API

↓

DriverManager

↓

JDBC Driver

↓

Database

JDBC allows Java applications to connect to and communicate with databases.

Database Table

```
CREATE DATABASE ecommerce;
USE ecommerce;
CREATE TABLE products(
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100),
   price DOUBLE,
   quantity INT
);
```

JDBC Servlet

```
import java.io.*;
import java.sql.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class ProductServlet extends HttpServlet {
   String url = "jdbc:mysql://localhost:3306/ecommerce";
   String user = "root";
   String password = "root";
   protected void doPost(HttpServletRequest req,
                         HttpServletResponse res)
                         throws IOException {
      String name = req.getParameter("name");
      double price = Double.parseDouble(req.getParameter("price"));
      int quantity = Integer.parseInt(req.getParameter("quantity"));
      try {
         Class.forName("com.mysql.cj.jdbc.Driver");
         Connection con =
            DriverManager.getConnection(url, user, password);
         String sql =
            "INSERT INTO products(name,price,quantity) VALUES(?,?,?)";
         PreparedStatement ps = con.prepareStatement(sql);
         ps.setString(1, name);
         ps.setDouble(2, price);
         ps.setInt(3, quantity);
         ps.executeUpdate();
         res.getWriter().println("Product Added Successfully");
```

```
            ps.close();
            con.close();
        } catch (Exception e) {
            res.getWriter().println("Error: " + e.getMessage());
        }
    }
}
```

Retrieving Products

```
Statement st = con.createStatement();
ResultSet rs =
    st.executeQuery("SELECT * FROM products");
while (rs.next()) {
    System.out.println(
        rs.getInt("id") + " " +
        rs.getString("name") + " " +
        rs.getDouble("price"));
}
```

JDBC Steps

1. Load Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

2. Create Connection

```
DriverManager.getConnection(url, user, password);
```

3. Create Statement/PreparedStatement
4. Execute SQL query
5. Process ResultSet
6. Close connection and resources

Exception Handling

JDBC uses try-catch to handle errors such as SQLException, ClassNotFoundException, and invalid input